

PYTHON FOUNDATION CHALLENGE

Revision + Progression

Objective: strengthen foundations, expose gaps, and develop problem-solving ability before OOP.

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean up the code afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.

Part A — Previous Questions: Revision

Redo these from memory. The purpose is to check retention before moving into the new challenges.

1. Simple Calculator

Suggested time: 10–15 minutes

Problem

Create a calculator that accepts two numbers and an operator, then performs the requested calculation. The calculator must support +, -, *, and /.

Examples

Input: 10 | Operator: * | Input: 5 → Result: 50

Input: 20 | Operator: / | Input: 4 → Result: 5

Input: 10 | Operator: % | Input: 2 → Output: Invalid operator

Input: 10 | Operator: / | Input: 0 → Output: Cannot divide by zero

Constraints

- Use input() and convert the numbers to an appropriate numeric type.
- Use if / elif / else.
- Handle invalid operators.
- Handle division by zero.
- Do not use eval().
- Do not import libraries.

2. Palindrome

Suggested time: 15–20 minutes

Problem

Write a program that determines whether a string reads the same forwards and backwards.

Examples

Input: racecar → Output: Palindrome

Input: madam → Output: Palindrome

Input: python → Output: Not a palindrome

Input: hello → Output: Not a palindrome

Constraints

- Use a loop.
- Do not use `[::-1]`.
- Do not use `.reverse()`.
- You may use `len()`.
- Assume the input contains lowercase letters only.
- Do not import libraries.

Optional Stretch

Try comparing characters from opposite ends of the string instead of simply creating a reversed string.

3. Student Grades

Suggested time: 20–25 minutes

Problem

Given a list of students and their scores, calculate each student's average and assign a grade.

Examples

```
students = [["Samuel", 80, 75, 90], ["David", 55, 60, 50], ["Mary", 35, 40, 30], ["John", 65, 70, 68]]
```

Samuel → Average: 81.67 → Grade: A

David → Average: 55.00 → Grade: C

Mary → Average: 35.00 → Grade: F

John → Average: 67.67 → Grade: B

Grading System

70–100 → A

60–69 → B

50–59 → C

45–49 → D

40–44 → E

0–39 → F

Constraints

- Use a loop to process every student.
- Calculate the averages yourself.
- Do not use `sum()`.
- Do not use `statistics.mean()`.
- Use `if / elif / else` for grading.

- Do not hardcode the final grades.
- Adding another student should not require changing the processing logic.

Optional Stretch

Create `calculate_grade(average)` as a bonus after the basic version works.

Part B — New Questions: Progression

These are new problems designed to increase the difficulty and reveal gaps in core Python problem-solving skills.

4. Second Largest

Suggested time: 15–20 minutes

Problem

Given a list of numbers, find the second-largest unique number.

Examples

Input: [10, 5, 8, 20, 15] → Output: 15

Input: [4, 9, 2, 9, 7] → Output: 7

Constraints

- Do not use `sort()`.
- Do not use `sorted()`.
- Do not use `max()` twice.
- Use loops and conditionals.
- Duplicate values must not affect the result.
- Assume the list contains at least two different numbers.

5. Word Frequency

Suggested time: 15–20 minutes

Problem

Given a sentence, count how many times each word appears. Store the results in a dictionary.

Examples

Input: "python is fun and python is powerful"

Output:

python: 2

is: 2

fun: 1

and: 1

powerful: 1

Constraints

- Use a dictionary.
- Use a loop.
- Do not use `collections.Counter`.
- Treat uppercase and lowercase as the same.
- You may use `.lower()`.
- You may use `.split()`.

- Do not hardcode the words.

Optional Stretch

Stretch: handle punctuation so that "python" and "python!" are treated as the same word.

6. Two Sum

Suggested time: 20–25 minutes

Problem

Given a list of numbers and a target number, find two numbers whose sum equals the target. Return the first valid pair you find.

Examples

Numbers: [2, 7, 11, 15] | Target: 9 → 2 + 7 = 9

Numbers: [3, 8, 4, 6] | Target: 10 → 4 + 6 = 10

Constraints

- Use loops.
- Do not use sum().
- Do not use a library.
- Do not hardcode the answer.
- The two numbers must come from different positions in the list.
- Assume at least one valid pair exists.

Optional Stretch

Stretch: return the indexes instead of the numbers. For [2, 7, 11, 15] with target 9, return [0, 1].

Progression

The challenge moves from basic syntax and logic → strings and loops → combining foundations → list algorithms → dictionaries → multi-step problem solving.

Time Guide

Section	Questions	Suggested Time
Revision	1–3	10–15 min total
New progression	4–6	45–50 min total
Overall	1–6	About 60 min